웹 프로그래밍

3. Mongoose 기초 및 To-Do API 만들기

류영표 강사
ryp1662@gmail.com

Copyright ⓒ “Youngpyo Ryu” All Rights Reserved.
This document was created for the exclusive use of “Youngpyo Ryu”.
It must not be passed on to third parties except with the explicit prior consent of “Youngpyo Ryu”.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com

학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- LG 헬로비전 DX 데이터 스쿨 (4기)
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 공공데이터 청년 인턴 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

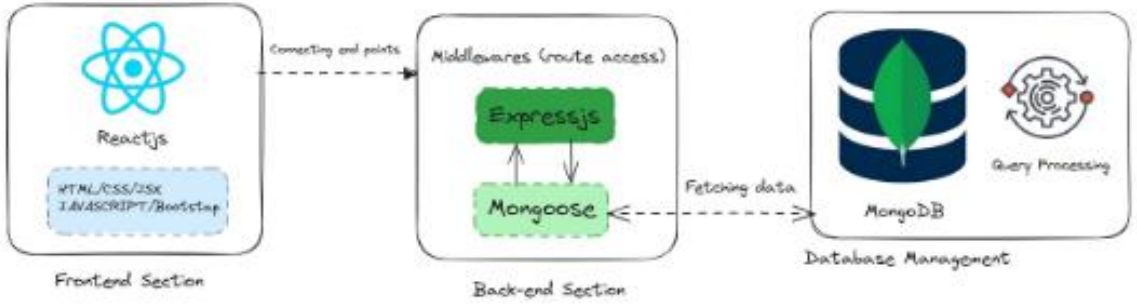
아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

Object Data Modeling (ODM)

Mongoose는 MongoDB의 데이터를 JavaScript 객체로 변환하고 관리할 수 있게 해주는 라이브러리.

단순한 DB 드라이버를 넘어, 데이터의 구조를 정의하고 비즈니스 로직을 코드 수준에서 캡슐화할 수 있도록 돕습니다.



** 캡슐화는 객체의 내부 데이터와 구현을 숨기고, 필요한 기능만 공개된 인터페이스로 제공해 외부가 직접 내부를 건드리지 못하게 하는 것이다.

Mongoose

스키마 정의

NoSQL의 자유로움 속에서도 데이터의 엄격한 구조를 부여할 수 있습니다.

유효성 검사

내장된 벨리데이터를 통해 잘못된 데이터가 DB에 쌓이는 것을 방지합니다.

편리한 쿼리

Promise 지원 및 직관적인 메서드 체이닝으로 쿼리 작성이 간편해집니다.

- Promise는 “지금 당장은 끝나지 않는 작업(비동기)의 결과를 나중에 주겠다는 약속”을 **객체로 표현한 것**
- 직관적인 메서드 체이닝은 여러 메서드를 점(.)으로 이어 한 흐름처럼 순차 실행하게 만드는 방식이다.

데이터의 무결성과 구조

설계 기반의 개발

Mongoose는 'Schema-first' 접근 방식을 취합니다. 이는 데이터베이스의 유연성을 유지하면서도 애플리케이션 레벨에서의 예측 가능성을 높여줍니다.

복잡한 프로젝트일수록 구조화된 데이터 관리가 성능과 유지보수의 핵심이 됩니다.

스키마(Schema)

- **필드 정의:** 데이터의 이름과 타입을 명확히 지정합니다 (String, Number, Date 등).
- **기본값(Default):** 필드값이 없을 때 적용될 기본 동작을 설정할 수 있습니다.
- **유효성 검사:** 필수 여부(required)나 정규표현식 매칭 등을 스키마 레벨에서 처리합니다.
- **관계 설정:** ObjectId를 활용하여 다른 컬렉션과의 참조 관계를 정의합니다.

모델(Model)

- **모델의 정의:** 스키마를 통해 생성된 컴파일된 클래스입니다.
- **문서(Document) 생성:** 모델 인스턴스는 하나의 MongoDB 문서를 나타냅니다.
- **정적 메서드:** 모델 자체에서 호출되는 메서드(예: find)를 통해 데이터 세트를 관리합니다.
- **인스턴스 메서드:** 특정 문서 객체에서 수행되는 개별 로직을 정의할 수 있습니다.

데이터 생성 및 조회(C & R)

Create (생성)

save() 메서드나 create() 정적 메서드를 사용하여 새 데이터를 저장합니다.

```
user . save )
```

Read (조회)

find(), findOne(), findById()를 통해 강력한 쿼리를 수행합니다.

필터링, 정렬(sort), 페이징(limit/skip) 기능이 내장되어 있습니다.

데이터 수정 및 삭제(U & D)

Update (수정)

updateOne(), findByIdAndUpdate() 등을 사용합니다.

수정 후의 문서를 반환받거나 검증 옵션을 추가할 수 있습니다.

Delete (삭제)

deleteOne(), findByIdAndDelete()를 통해 데이터를 안전하게 제거합니다.

성능 최적화를 위해 원자적 작업을 수행합니다.

자동화된 코드 검증

0%

Data Corruption

엄격한 유효성 검사

Mongoose를 사용하면 애플리케이션 로직에서 일일이 데이터를 검사할 필요가 없습니다.

스키마 수준에서 required: true, minlength: 5와 같은 제약 조건을 선언하면, 규칙에 어긋나는 데이터는 저장이 거부됩니다.

Mongoose

- MongoDB용 ODM(Object Document Mapping) 라이브러리
- MongoDB 문서(Document) ↔ JavaScript 객체(Object) 를 연결해, DB 데이터를 코드에서 객체처럼 다루게 해줌
- Mongoose는 DB 작업이 바로 끝나지 않기 때문에, **끝나면 다음 동작을 이어서 처리할 수 있게** then/catch나 async/await로 결과를 기다리며 코드를 작성할 수 있다.

ODM이 해주는 일(핵심)

- DB에서 가져온 문서를 JS 객체로 변환
- JS 객체를 DB 문서로 저장/수정

장점

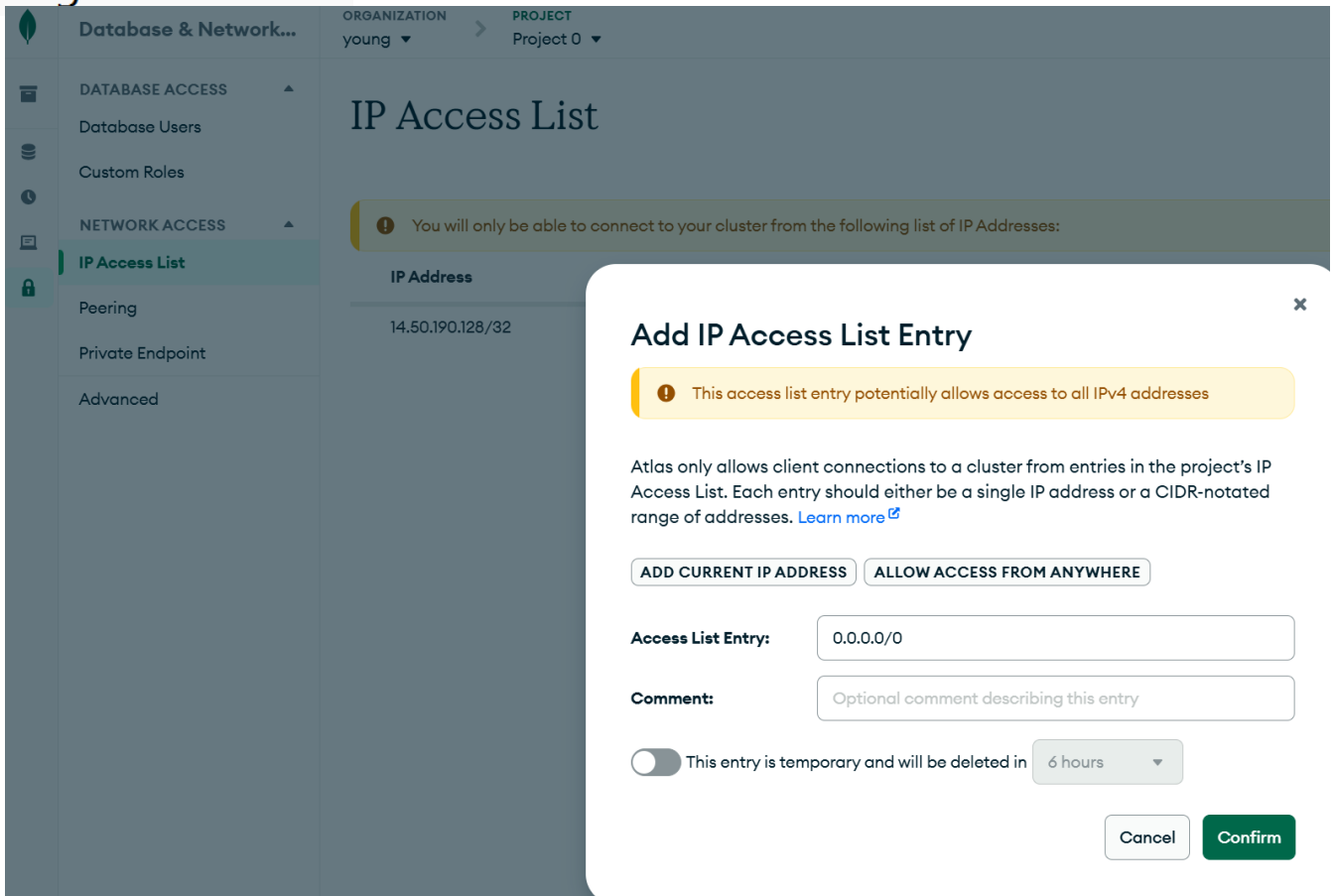
- 스키마 정의로 데이터 구조를 명확히 관리(필수값, 타입, 범위 등 검증)
- 스키마를 기반으로 모델(Model) 을 만들고, 모델로 CRUD를 간단히 수행
- createdAt / updatedAt 같은 공통 필드를 자동 관리 가능
- 코드가 모델 중심으로 정리되어 유지보수에 유리

Mongoose: 그냥 저장에서 규칙 있는 저장으로

- MongoDB는 그냥 들어가고, Mongoose는 규칙(스키마)으로 막아줌

```
mkdir mongoose-proof
cd mongoose-proof
npm init -y
npm i mongodb mongoose dotenv
```

dotenv는 프로젝트 루트의 .env 파일에 적어둔 설정값(환경변수)을 Node.js 코드에서 읽어오게 해주는 라이브러리



Mongoose: 그냥 저장에서 규칙 있는 저장으로

- MongoDB는 그냥 들어가고, Mongoose는 규칙(스키마)으로 막아줌

File Edit Selection View Go Run Terminal Help

EXPLORER

MONGOOSE-PROOF
- node_modules
- .env
- 01_native.js
- 02_mongoose.js
- package-lock.json
- package.json

01_native.js

```javascript
require("dotenv").config();
const { MongoClient } = require("mongodb");

async function main() {
  const uri = process.env.MONGODB_URI;
  if (!uri) throw new Error("MONGODB_URI가 .env에 없습니다.");

  const client = new MongoClient(uri);
  await client.connect();

  // DB 이름을 지정하지 않음 -> Atlas 기본 DB로 저장(보통 test)
  const db = client.db();
  const col = db.collection("t1_native");

  await col.deleteMany({});

  await col.insertOne({
    age: "not number",
    role: "godmode",
    extra: 123,
  });

  const stored = await col.findOne({});
  console.log("\n[Native Driver] 저장 결과 (그대로 들어가야 함):");
  console.log(stored);

  await client.close();
}

main().catch((e) => {
  console.error("ERROR:", e.message);
  process.exit(1);
});
```

```
C:\Users\USER\Documents\test\mongoose-proof>node 01_native.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: suppress all logs with { quiet: true }

[Native Driver] 저장 결과 (그대로 들어가야 함):
{
  _id: new ObjectId('6967b5170883ee249c956f05'),
  age: 'not number',
  role: 'godmode',
  extra: 123
}
```

Mongoose: 그냥 저장에서 규칙 있는 저장으로

- MongoDB는 그냥 들어가고, Mongoose는 규칙(스키마)으로 막아줌

EXPLORER

MONGOOSE-PROOF
- node_modules
- .env
- 01_native.js
- 02_mongoose.js
- package-lock.json
- package.json

OUTLINE

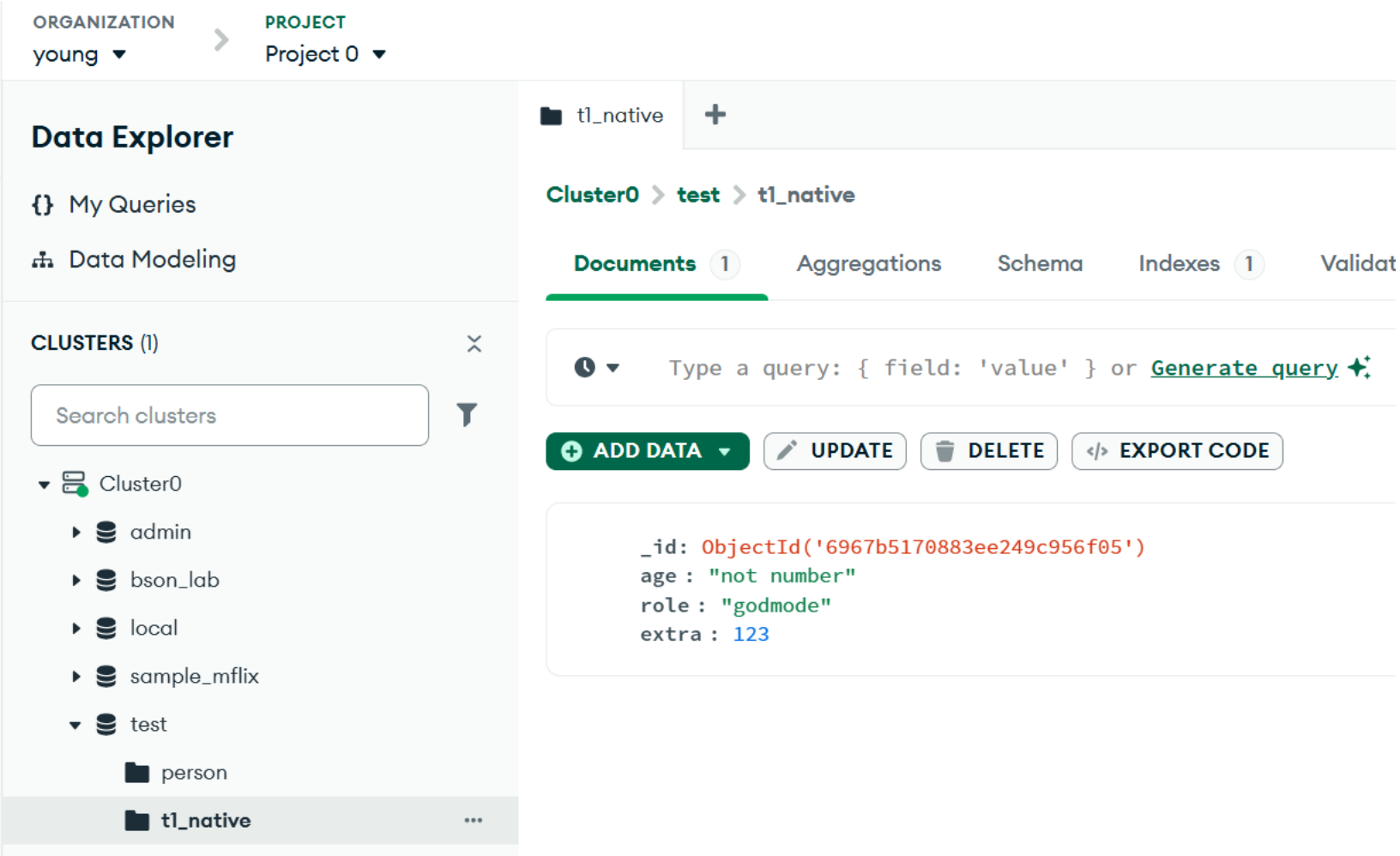
02_mongoose.js

```javascript
require("dotenv").config();
const mongoose = require("mongoose");

async function main() {
  const uri = process.env.MONGODB_URI;
  if (!uri) throw new Error("MONGODB_URI가 .env에 없습니다.");

  await mongoose.connect(uri);

  // strict: "throw" => 스키마에 없는 필드(extra)는 에러로 차단
  const User = mongoose.model(
    "UserMin",
    new mongoose.Schema(
      { age: { type: Number, required: true } },
      { strict: "throw" }
    ),
    "t1_mongoose"
  );

  await User.deleteMany({});

  console.log("\n[Mongoose] 저장 시도 (막혀야 정상):");
  try {
    await User.create({ age: "not number", extra: 123 });
    console.log("Unexpected: saved (이 줄이 나오면 목표와 다름)");
  } catch (e) {
    console.log("Blocked:", e.message);
  }

  await mongoose.disconnect();
}

main().catch((e) => {
  console.error("ERROR:", e.message);
  process.exit(1);
});
```

```
C:\Users\USER\Documents\test\mongoose-proof>node 02_mongoose.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🔒 prevent building .env in docker: https://dotenvx.com/prebuild

[Mongoose] 저장 시도 (막혀야 정상):
Blocked: Field `extra` is not in schema and strict mode is set to throw.
```

Mongoose: 그냥 저장에서 규칙 있는 저장으로

- MongoDB는 그냥 들어가고, Mongoose는 규칙(스키마)으로 막아줌



01. Express 서버 + 정적 페이지(브라우저 시각 확인)

```
npm init -y
npm i express mongoose dotenv
mkdir models routes public
```

EXPLORER
- TODO-LAB
  - models
  - node_modules
  - public
  - routes
  - .env
  - package-lock.json
  - package.json

EXPLORER
- TODO-LAB
  - models
  - node_modules
  - public
    - 01_index.html
  - routes
  - .env
  - 01_app.js
  - package-lock.json
  - package.json

01_app.js

```
01_app.js > ...
1  require("dotenv").config();
2  const express = require("express");
3
4  const app = express();
5  app.use(express.static("public"));
6
7  app.get("/health", (req, res) => res.send("OK"));
8
9  const PORT = process.env.PORT || 3000;
10 app.listen(PORT, () => {
11   console.log(`01 server: http://localhost:${PORT}`);
12 });
```

EXPLORER
- TODO-LAB
  - models
  - node_modules
  - public
    - index.html
  - routes
  - .env
  - 01.app.js
  - package-lock.json
  - package.json

index.html

```
public > index.html > ...
1  <!doctype html>
2  <html lang="ko">
3  <head>
4    <meta charset="utf-8" />
5    <meta name="viewport" content="width=device-width,initial-scale=1" />
6    <title>01 - Server OK</title>
7  </head>
8  <body>
9    <h1>01. Express 서버가 켜졌습니다</h1>
10   <p>이 페이지가 보이면 성공입니다.</p>
11 </body>
12 </html>
```

```
C:\Users\USER\Documents\test\todo-lab>node 01.app.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: prevent building .env in docker: https://dotenvx.com/prebuild
01 server: http://localhost:3000
```

- 서버만 띄우는 작업

localhost:3000/health

OK

01 - Server OK

localhost:3000/01_index.html

01. Express 서버가 켜졌습니다

이 페이지가 보이면 성공입니다.

02. MongoDB Altas 연결 + “서버 콘솔 표 출력” 확인

- Altas 연결 성공 로그
- “Todo 1개 저장“을 서버에서 수행하고, 서버 콘솔에 console.table()로 결과 출력(02단계에서는 아직 입력을 받지 않음)ㅌ

EXPLORER

TODO-LAB
- models
- node_modules
- public
- routes
- .env
- 01_app.js

02_app.js

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose")

const app = express();
app.use(express.static("public"));
```

```
C:\Users\USER\Documents\test\todo-lab>node 02_app.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🔐 prevent committing .env to code: https://dotenvx.com/precommit
MongoDB connected
02 server: http://localhost:3000
```

localhost:3000/health

localhost:3000/health

OK

03. 브라우저 입력 -> API 저장 -> 리스트 즉시 갱신

- 서버 + DB + API

```
EXPLORER
TODO-LAB
> models
> node_modules
> public
> routes
.env
01_app.js
```

03_app.js

```
1  require("dotenv").config();
2  const express = require("express");
3  const mongoose = require("mongoose");
4
5  const app = express();
6  app.use(express.json());
7  app.use(express.static("public"));
```

- 화면

```
EXPLORER
TODO-LAB
> models
> node_modules
v public
  01_index.html
  03_index.html
```

03_index.html

```
1  <!doctype html>
2  <html lang="ko">
3  <head>
4    <meta charset="utf-8" />
5    <meta name="viewport" content="w
6    <title>03 - Todo (API + MongoDB)
```

- 브라우저 동작

File Edit Selection View Go Run Terminal Help

```
EXPLORER
TODO-LAB
> models
> node_modules
v public
  01_index.html
```

03_app.js

```
1  const form = document.getElementById("form");
2  const input = document.getElementById("content");
3  const list = document.getElementById("list");
4  const statusEl = document.getElementById("status");
5
```

```
C:\Users\USER\Documents\test\todo-lab>node 03_app.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: write to custom object with { processEnv: myObject }
MongoDB connected
03 server: http://localhost:3000/03_index.html
```

03 - Todo (API + MongoDB)

localhost:3000/03_index.html

03. Todo

할 일을 입력하세요 | 추가

총 0개

리스트

04. 삭제(Delete) 추가

- 03_app.js에 삭제 기능 추가

EXPLORER

TODO-LAB
- models
- node_modules
- public
  - 01_index.html
  - 03_app.js

04_app.js

```
require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");

const app = express();
```

- 브라우저: 03 + 삭제 버튼 추가

EXPLORER

TODO-...
- models
- node_modules
- public
  - 01_index.html
  - 03_app.js
  - 03_index.html
  - 04_app.js
  - 04_index.html

public > 04_app.js > ...

```
const form = document.getElementById("form");
const input = document.getElementById("content");
const list = document.getElementById("list");
const statusEl = document.getElementById("status");

/* ========================
 * (03단계) 목록 가져오기
 * - GET /api/todos
 * - 가져온 데이터를 render()로 화면에 그림
```

```
C:\Users\USER\Documents\test\todo-lab>node 04_app.js
[dotenv@17.2.3] injecting env (2) from .env -- tip: 🔐 prevent building .env in docker: https://dotenvx.com/prebuild
MongoDB connected
04 server: http://localhost:3000/04_index.html
```

04 - Todo (Delete)

localhost:3000/04_index.html

04. Todo (삭제 기능)

할 일을 입력하세요 | 추가

항목 오른쪽에 **삭제** 버튼이 나타나야 합니다. (버튼은 JS가 목록을 그릴 때 자동으로 생성됩니다.)

총 1개

리스트

- to do list (2026. 1. 15. 오전 8:37:00) 삭제

Thank you.

웹 프로그래밍
ryp1662@gmail.com

Copyright ⓒ “Youngpyo Ryu” All Rights Reserved.
This document was created for the exclusive use of “Youngpyo Ryu”.
It must not be passed on to third parties except with the explicit prior consent of “Youngpyo Ryu”.